

AULA 3 · 25 de agosto · U1, Linguagens e Paradigmas

Tipagem, execução e memória

Retomando os três eixos com profundidade, casos reais e medição em laboratório

CODING: LINGUAGENS E TÉCNICAS

Prof. Dr. Sebastião Rogério · Faculdade SENAC Pernambuco, ADS 2026.2



0.1 + 0.2 não dá 0.3.

Não é bug de linguagem nem erro de quem escreveu. É consequência de como o computador representa números com vírgula, e vale para praticamente toda linguagem que usa o padrão IEEE 754.

Verifique no console do navegador antes de continuar a aula.

Por que voltar a este assunto

Os três eixos já apareceram na abertura da disciplina, de forma resumida.

A revisita tem um motivo prático. Aqueles conceitos foram apresentados quando ainda não havia código escrito para associá-los, e conceito sem uso não fixa. Agora existe um projeto em andamento, e cada eixo tem consequência direta sobre decisões que as equipes vão tomar nas próximas semanas.

A parte conceitual desta aula é deliberadamente rápida. O tempo está concentrado em três coisas: casos reais em que esses conceitos custaram caro, o comportamento que se pode observar na própria máquina, e um laboratório de medição comparando duas linguagens.



A ideia de fundo, em uma frase

Toda abstração vaza, como escreveu Joel Spolsky. A linguagem promete que você não precisa pensar em bits, e cumpre a promessa na maior parte do tempo. Os eixos desta aula descrevem exatamente onde ela deixa de cumprir.

Os três eixos, em revisão rápida

1

Tipagem

Quando o erro de tipo aparece, e quanto a linguagem improvisa para não falhar.

Pergunta de fundo: a linguagem confia mais no compilador ou no programador?

2

Execução

Como o texto que você escreve vira instrução que o processador entende.

Pergunta de fundo: o que acontece entre salvar o arquivo e ver o resultado?

3

Memória

Quem devolve ao sistema o espaço que o programa deixou de usar.

Pergunta de fundo: você, um coletor automático, ou o próprio compilador?

O que muda em relação à primeira passagem

Antes, os eixos serviam para comparar linguagens. Agora servem para explicar defeitos concretos: um valor monetário errado, uma acentuação quebrada, um processo que incha até cair.

BLOCO 1

Tipagem

Quando o erro aparece, e o que a linguagem faz quando os tipos não combinam.



Eixo A: estática ou dinâmica

A diferença não é se o erro acontece. É em que momento ele é descoberto.

DINÂMICA, o tipo é verificado ao executar

```
def total(preco, qtd):  
    return preco * qtd  
  
total(19.90, 3)      # 59.7, correto  
  
total(19.90, "3")    # nao falha aqui!  
# "3" repetido 19 vezes, resultado absurdo
```

ESTÁTICA, o tipo é verificado antes de executar

```
function total(  
    preco: number,  
    qtd: number  
): number {  
    return preco * qtd;  
}  
  
total(19.90, "3"); // nem compila
```



Este exemplo não é hipotético

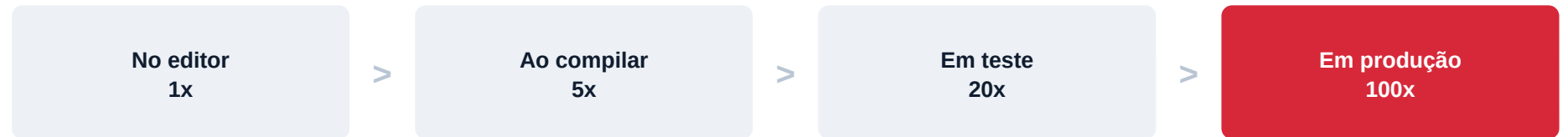
Todo campo de formulário HTML chega ao servidor como texto, inclusive os campos numéricos. Esquecer a conversão é uma das origens mais comuns de defeito em sistemas web, e em linguagem dinâmica esse defeito só aparece quando um usuário real tropeça nele.

O custo de descobrir tarde

O mesmo defeito custa valores muito diferentes conforme a fase em que é encontrado.

Custo relativo

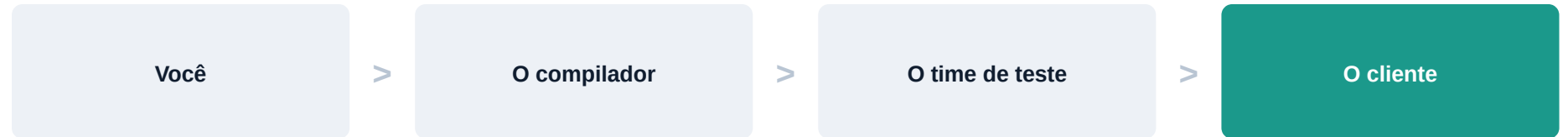
ordem de grandeza



A proporção exata varia conforme o estudo, mas a direção é consensual na engenharia de software desde os anos 1980.

Quem descobre

em cada fase



Tipagem estática empurra a descoberta para as duas primeiras colunas. Esse é o argumento central a favor dela.

O outro lado do argumento

Tipagem estática cobra tempo de escrita e cerimônia. Em um script de 40 linhas que roda uma vez, esse custo não se paga. A escolha depende do tempo de vida do código.

Eixo B: forte ou fraca

Quando os tipos não combinam, a linguagem reclama ou tenta adivinhar o que você quis dizer?

FRACA, JavaScript improvisa para não falhar

```
"3" + 1      // "31" virou texto
"3" - 1      // 2    virou numero
[] + {}      // "[object Object]"
[] == false  // true

// nenhuma dessas linhas gera erro
```

FORTE, Python prefere reclamar a adivinhar

```
"3" + 1
# TypeError: can only concatenate
#          str (not "int") to str

int("3") + 1  # 4, voce foi explicito

# o erro aparece na linha do problema
```



Os dois eixos são independentes

Python é dinâmica e forte. JavaScript é dinâmica e fraca. Java é estática e forte. C é estática e relativamente fraca, porque permite tratar um ponteiro como número inteiro sem reclamar.

A comparação que confunde meio mundo

JavaScript tem dois operadores de igualdade, e a diferença entre eles é justamente a conversão automática de tipo.

== compara depois de converter

```
0 == "0"      // true
0 == ""       // true
"" == "0"     // false (!)
```

```
null == undefined // true
[] == false        // true
```

=== compara sem converter

```
0 === "0"     // false
0 === ""      // false
"" === "0"    // false
```

```
null === undefined // false
[] === false        // false
```



Repare na terceira linha da esquerda

Zero é igual a "0", zero é igual a "", mas "0" não é igual a "". A igualdade deixa de ser transitiva, o que quebra uma propriedade que qualquer pessoa assume sem pensar. Por isso a recomendação profissional é usar sempre a comparação estrita.

Quando o tipo é pequeno demais

Todo tipo numérico tem um valor máximo, e o que acontece ao ultrapassá-lo depende da linguagem.

Em linguagens com inteiros de tamanho fixo, o contador dá a volta e vira negativo. Foi o que aconteceu no acidente do Ariane 5, e é o mesmo fenômeno que obrigou o YouTube a migrar seu contador de visualizações para um tipo maior quando um vídeo ultrapassou o limite de um inteiro de 32 bits.

Python é uma exceção conveniente: seus inteiros crescem conforme a necessidade, limitados apenas pela memória disponível. Isso evita a classe inteira de erro, ao custo de operações mais lentas.

A consequência prática aparece na modelagem: escolher o tipo de um campo de banco de dados é decidir qual o maior valor que aquele campo poderá receber ao longo de toda a vida do sistema.

A matriz completa

Linguagem	Estática ou dinâmica	Forte ou fraca	Consequência prática mais visível
Python	Dinâmica	Forte	Escreve rápido, e o erro de tipo estoura em execução com mensagem clara
JavaScript	Dinâmica	Fraca	Quase nada gera erro, e o defeito aparece como resultado errado, não como exceção
TypeScript	Estática	Forte	O editor acusa antes de rodar, ao custo de declarar tipos
Java	Estática	Forte	Compilação barra o erro, com bastante código cerimonial
C	Estática	Fraca	Compila, mas permite conversões perigosas que o programador precisa vigiar
Rust	Estática	Forte	O compilador é exigente e recusa muito código, inclusive código que funcionaria



Este quadro reaparece na avaliação

A exigência não é decorar a tabela. É conseguir preenchê-la sozinho diante de uma linguagem nunca vista, a partir de dois ou três experimentos.

Dois acidentes causados por tipo

Erro de tipo não é assunto de sala de aula. Dois dos casos mais estudados da engenharia de software são exatamente isso.

A

Ariane 5, voo inaugural, 1996

Trinta e sete segundos após a decolagem, o foguete se autodestruíu.

Causa: um valor de velocidade horizontal, guardado como número de ponto flutuante de 64 bits, foi convertido para um inteiro de 16 bits. O valor não cabia. A conversão gerou uma exceção não tratada e o sistema de navegação parou.

O trecho de código era herdado do Ariane 4, onde aquele valor nunca ficava grande o bastante para estourar.

B

Mars Climate Orbiter, 1999

A sonda entrou na atmosfera de Marte em altitude errada e foi destruída.

Causa: um módulo produzia valores de impulso em libra-força por segundo, e o outro os consumia esperando newton por segundo. Nenhum dos dois estava errado isoladamente.

Nenhum tipo do sistema carregava a informação de unidade, então nada no código podia acusar a incompatibilidade.

O que os dois casos têm em comum

Em ambos, o computador fez exatamente o que o código mandou. O que faltou foi um tipo capaz de expressar a restrição, e uma verificação que a checasse antes da operação valer.

O erro de um bilhão de dólares

Tony Hoare criou a referência nula em 1965 e, décadas depois, chamou publicamente a decisão de seu erro de um bilhão de dólares.

O problema é simples de enunciar: quando qualquer variável pode conter nulo, o sistema de tipos deixa de dizer a verdade. A assinatura promete um cliente, mas pode entregar nada, e o programa só descobre isso quando tenta usar o valor.

Em Python, o sintoma é `AttributeError` sobre `NoneType`. Em Java, é `NullPointerException`. É provavelmente a exceção mais comum da história da programação, e ela existe porque o tipo não distinguia entre um valor presente e a ausência de valor.

Linguagens mais recentes tratam a ausência como um tipo próprio, que o compilador obriga a verificar antes do uso. Em Python, a anotação `Optional` cumpre esse papel para as ferramentas de análise.

Tony Hoare, palestra `Null References: The Billion Dollar Mistake`, QCon London, 2009.

O meio-termo: tipagem gradual

Desde a versão 3.5, Python permite anotar tipos. O interpretador ignora as anotações, mas as ferramentas não.

SEM ANOTAÇÃO, o que este código espera?

```
def aplicar_desconto(p, d):  
    return p - p * d  
  
# d e 0.1 ou 10?  
# p aceita string?  
# o que a funcao devolve?  
  
# so lendo o corpo se descobre
```

COM ANOTAÇÃO, a assinatura já responde

```
def aplicar_desconto(  
    preco: float,  
    desconto: float # fracao de 0 a 1  
) -> float:  
    return preco - preco * desconto  
  
# o editor completa e avisa  
# mypy acusa erro antes de rodar
```



Por que isso importa para o projeto do semestre

Anotação de tipo é documentação que não desatualiza, porque a ferramenta acusa quando ela deixa de ser verdadeira. Comentário mente com o tempo, assinatura de tipo não. A disciplina vai usar anotações a partir da unidade de orientação a objetos.

O caso do dinheiro

0,1 + 0,2

resulta em
0,30000000000000004

IEEE 754

o padrão usado por
quase toda linguagem

nunca

use ponto flutuante
para valor monetário



Por que isso acontece

Ponto flutuante representa números em base dois. Assim como um terço não tem representação decimal exata e vira 0,333..., um décimo não tem representação binária exata. O computador guarda o valor mais próximo que consegue, e o erro se acumula a cada operação.



O que se usa no lugar

Em Python, o tipo Decimal. Em Java, BigDecimal. Uma alternativa comum e simples é guardar tudo em centavos, como número inteiro, e dividir apenas na hora de exibir. Todo sistema financeiro sério faz uma dessas duas coisas.

BLOCO 2

Execução

O caminho entre o arquivo que você salva e a instrução que o processador executa.

2

O processador não entende Python

Nenhum processador entende Python, Java ou JavaScript. Ele entende sequências de bytes que representam operações elementares: mover um valor, somar, comparar, saltar para outro endereço.

Tudo que uma linguagem de alto nível oferece precisa ser traduzido para isso em algum momento. A pergunta que separa as famílias de linguagens é quando essa tradução acontece e quem a executa.



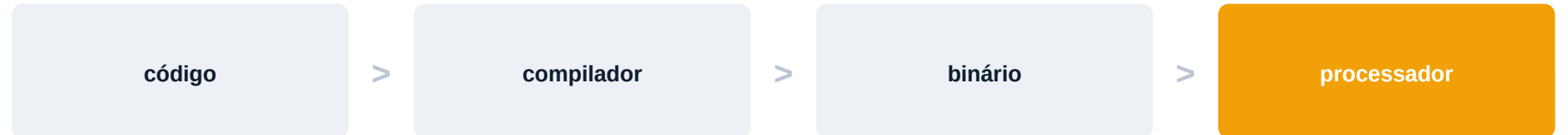
Um detalhe que costuma confundir

Compilada e interpretada não são propriedades da linguagem, e sim da implementação. Existe interpretador de C e existe compilador de Python. Na prática, porém, cada linguagem tem uma implementação dominante, e é dela que se fala no dia a dia.

Os três caminhos

Compilada

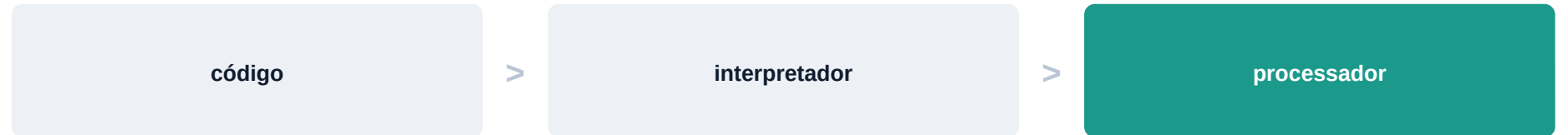
C, C++, Go, Rust



A tradução acontece uma vez, antes de distribuir. O binário resultante roda rápido, mas só serve para o sistema operacional e a arquitetura para os quais foi gerado.

Interpretada

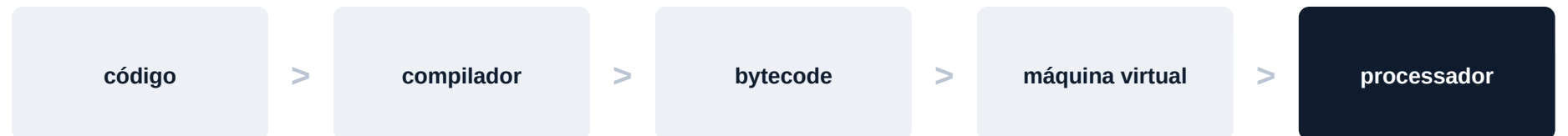
Python, PHP, Ruby



A tradução acontece a cada execução, linha a linha. O ciclo de edição é imediato, a execução é mais lenta, e a máquina de destino precisa ter o interpretador instalado.

Híbrida

Java, C#, Kotlin



Compila para um formato intermediário que uma máquina virtual executa. É o que permite escrever uma vez e rodar em qualquer lugar, ao custo de subir a máquina virtual.

Texto também é um tipo, e ele tem armadilhas

Todo caractere precisa virar bytes em algum momento. A regra usada nessa conversão é a codificação.

O SINTOMA QUE TODO MUNDO JÁ VIU

```
# arquivo gravado em UTF-8
# lido como se fosse Latin-1

Joao da Silva      -> JoÃ£o da Silva
Coracao            -> CoraÃ§Ã£o

# nenhum erro e levantado
# o dado simplesmente sai errado
```

A PREVENÇÃO

```
# declarar a codificacao sempre
open("dados.csv",
      encoding="utf-8")

# no banco de dados, conferir
# a codificacao da tabela

# padrao seguro hoje: UTF-8 em tudo
```



Por que isso aparece tanto em projeto brasileiro

Nomes com acento e cedilha tornam o problema visível já no primeiro cadastro. Em sistemas escritos apenas em inglês, o mesmo defeito pode passar meses despercebido antes de aparecer.

O caso do JavaScript: compilação em tempo de execução

JavaScript era interpretado puro até meados dos anos 2000, e por isso tinha fama de lento.

Os motores modernos, como o V8 do Chrome, fazem algo diferente: começam interpretando, observam quais trechos são executados muitas vezes e compilam apenas esses para código de máquina, durante a execução. A técnica se chama compilação just-in-time.

O resultado é que o mesmo código pode ficar mais rápido depois de alguns segundos de execução, porque o motor descobriu o que vale a pena otimizar. Isso explica por que medir desempenho em JavaScript exige rodar o trecho muitas vezes antes de cronometrar.



A abstração vazando, de novo

Um desenvolvedor que não sabe da existência do just-in-time mede o tempo da primeira execução, conclui que o código está lento e otimiza a parte errada. Esse é exatamente o tipo de erro que o texto de Spolsky descreve.

Consequências práticas de cada caminho

Situação do dia a dia	Compilada	Interpretada	Híbrida
Tempo entre salvar e ver o resultado	segundos a minutos	imediatos	segundos
Velocidade de execução	alta	baixa	alta após aquecer
Distribuir para outra máquina	binário por plataforma	exige interpretador	exige máquina virtual
Descobrir erro de sintaxe	ao compilar	ao chegar na linha	ao compilar
Adequada para script rápido	não	sim	nem tanto
Adequada para jogo ou sistema crítico	sim	não	em parte

A linha mais reveladora é a última

Nenhum jogo de grande porte é escrito em Python puro, e nenhum script de automação de dez linhas é escrito em C. A escolha segue a natureza da tarefa.

Por que Python é lento

A resposta não é uma só. São três decisões de projeto que se somam, e todas foram tomadas de propósito.

1

Tudo é objeto

Um número inteiro em C ocupa 4 bytes na memória. Em Python, ele é um objeto com contador de referências, ponteiro de tipo e o valor. Somar dois inteiros envolve muito mais trabalho do que somar dois inteiros.

2

A busca é feita em execução

Ao encontrar a chamada `obj.metodo()`, o interpretador precisa descobrir em execução onde esse método está, porque o tipo de `obj` pode mudar entre uma linha e outra. Uma linguagem estática resolve isso na compilação.

3

Um fluxo por vez no interpretador padrão

O interpretador de referência tem um mecanismo que impede que dois fluxos de execução rodem código Python ao mesmo tempo. Isso simplifica o interpretador e limita o ganho de usar múltiplos núcleos para cálculo puro.

Por que isso raramente é um problema na prática

As bibliotecas pesadas de Python, como NumPy e Pandas, são escritas em C por baixo. Python funciona como a linguagem que organiza a tarefa, e o trabalho numérico acontece em código compilado.

Uma consequência que se sente todo dia

O tempo entre alterar uma linha e ver o efeito define o ritmo de quem programa.

Em linguagem interpretada, esse tempo é praticamente zero: salva e roda. Isso favorece um estilo de trabalho exploratório, em que se testam hipóteses rapidamente, e é uma das razões da adoção de Python em análise de dados e em pesquisa.

Em projetos compilados de grande porte, esse tempo pode chegar a minutos, e isso muda o comportamento da equipe. Passa a valer a pena pensar mais antes de rodar, escrever mais testes automatizados e evitar mudanças especulativas. Não é melhor nem pior, é outro ritmo.

A criação da linguagem Go tem exatamente essa origem. O tempo de compilação dos sistemas em C++ dentro do Google havia se tornado um obstáculo de produtividade, e a linguagem foi projetada desde o início para compilar rápido.

BLOCO 3

Memória

Todo programa pede memória emprestada ao sistema. A pergunta é quem se lembra de devolver.



Dois lugares onde os dados moram

1

Pilha, ou stack

Guarda as variáveis locais de cada função. Cresce quando uma função é chamada e encolhe automaticamente quando ela termina.

É rápida e não exige gerenciamento. Em compensação é pequena, e uma recursão sem fim a estoura: é a origem do erro stack overflow.

2

Monte, ou heap

Guarda dados que precisam sobreviver à função que os criou, como listas, objetos e textos longos.

É grande e flexível, mas alguém precisa decidir quando o espaço pode ser reaproveitado. É aqui que as três estratégias a seguir divergem.

O nome do site mais famoso da profissão vem daí

Stack Overflow leva o nome do erro que acontece quando a pilha de chamadas de função excede o espaço reservado para ela.

Estratégia 1: gerenciamento manual

O programador pede o espaço e o programador devolve. Controle total, responsabilidade total.

C, o fluxo correto

```
int *v = malloc(100 * sizeof(int));
if (v == NULL) return -1;

// ... usar o vetor ...

free(v); // devolve ao sistema
v = NULL; // evita uso indevido
```

C, os três erros clássicos

```
// 1. esquecer o free
//    vazamento: o programa incha

// 2. chamar free duas vezes
//    corrompe a estrutura interna

// 3. usar depois do free
//    le lixo, ou abre brecha de segurança
```



Por que isso ainda importa em 2026

A equipe do Chromium relatou que cerca de 70% dos defeitos graves de segurança do navegador têm origem em falhas de segurança de memória. Foi essa conta que motivou a criação do Rust e as recomendações recentes de órgãos de segurança pública em favor de linguagens com memória segura.

Estratégia 2: coleta automática de lixo

Um componente do próprio ambiente descobre o que não é mais alcançável e recupera o espaço.

1

Como ele decide

A ideia central é alcançabilidade. Partindo das variáveis vivas, o coletor segue todas as referências. O que não for alcançado por esse caminho não pode mais ser usado por ninguém, e portanto pode ser recuperado.

2

O que se ganha

Uma classe inteira de defeitos desaparece. Não existe uso após liberação nem liberação dupla, e o programador dedica atenção ao problema de negócio em vez de à contabilidade de memória.

3

O que se paga

O coletor consome tempo de processador em momentos que o programador não escolhe. Em sistemas com exigência de resposta previsível, essas pausas são um problema real de projeto.

Coleta automática não elimina vazamento

Se o código mantém referência para algo que não usa mais, como um item nunca removido de uma lista global, o coletor considera o objeto alcançável e o mantém vivo. O vazamento passa a ser lógico, e não técnico.

Estratégia 3: posse verificada em compilação

Rust resolve o problema em um terceiro lugar: nem em execução, nem na disciplina do programador, e sim no compilador.

A linguagem impõe regras de posse. Cada valor tem exatamente um dono, e quando o dono sai de escopo o valor é liberado. Se o programa tenta usar um valor cuja posse foi transferida, o código simplesmente não compila.

O resultado é memória segura sem coletor e sem pausas imprevisíveis. O custo é uma curva de aprendizado notoriamente dura, porque o compilador recusa muito código, incluindo código que funcionaria perfeitamente bem.



A regra geral por trás dos três modelos

Toda linguagem escolhe onde pagar: em execução, na cabeça do programador ou na rigidez do compilador. Não existe opção sem custo, existe custo colocado em lugares diferentes.

Vazamento com coletor de lixo ligado

O coletor recupera o que ninguém alcança. O problema é quando o próprio código continua alcançando o que não precisa mais.

O CÓDIGO QUE VAZA

```
CACHE = {} # dicionario global

def buscar_cliente(cpf):
    if cpf not in CACHE:
        CACHE[cpf] = consultar_banco(cpf)
    return CACHE[cpf]

# nada nunca sai do CACHE
# apos um mes no ar, o processo cai
```

UMA CORREÇÃO POSSÍVEL

```
from functools import lru_cache

@lru_cache(maxsize=1000)
def buscar_cliente(cpf):
    return consultar_banco(cpf)

# limite declarado
# o mais antigo sai quando o limite estoura
```



Por que o coletor não resolve isso sozinho

Do ponto de vista dele, cada objeto no dicionário está perfeitamente alcançável a partir de uma variável global viva. O vazamento é lógico, não técnico, e nenhum mecanismo automático consegue adivinhar que aquele dado deixou de ser necessário.

Três experimentos para posicionar uma linguagem nova

Diante de qualquer linguagem desconhecida, estas três verificações dão a posição dela nos três eixos em poucos minutos.

1

Somar um texto com um número. Se der erro, a tipagem é forte. Se produzir um resultado estranho sem reclamar, é fraca.

2

Passar um argumento do tipo errado e observar quando o problema aparece. Antes de rodar indica tipagem estática, durante a execução indica dinâmica.

3

Procurar na documentação por uma função de liberar memória. Se existir, o gerenciamento é manual. Se houver menção a coletor de lixo, é automático. Se o compilador falar em posse ou empréstimo, é o modelo do Rust.

Esse roteiro reaparece na prova do primeiro processo avaliativo, aplicado a uma linguagem não vista em aula.

Os três eixos, seis linguagens

Linguagem	Tipagem	Execução	Memória	Onde ela é encontrada
Python	Dinâmica e forte	Interpretada	Coletor	IA, dados, automação, back-end
JavaScript	Dinâmica e fraca	Interpretada com JIT	Coletor	Navegador e servidores Node
TypeScript	Estática e forte	Transpilada para JS	Coletor	Front-end de porte, times grandes
Java	Estática e forte	Híbrida, JVM	Coletor	Bancos, sistemas corporativos, Android
C	Estática e fraca	Compilada	Manual	Sistemas operacionais, embarcados
Rust	Estática e forte	Compilada	Posse	Infraestrutura e sistemas críticos



Como usar este quadro no projeto

Ao encontrar uma linguagem nova, três experimentos bastam para posicioná-la: somar texto com número, verificar se existe etapa de build, e procurar uma função de liberar memória na documentação.

BLOCO 4

Laboratório

Medir em vez de supor. O mesmo algoritmo em duas linguagens, cronometrado e comparado.



Comparando Python e JavaScript na prática

O objetivo não é eleger um vencedor. É produzir dados próprios e explicar o que eles mostram.

1

Implementar o mesmo algoritmo nas duas linguagens

Somar os quadrados de todos os números de 1 até 10 milhões. Um arquivo .py e um arquivo .js, resolvendo exatamente o mesmo problema.

2

Cronometrar as duas execuções

Em Python, usar o módulo time. Em JavaScript, usar console.time e console.timeEnd. Rodar cada uma três vezes e registrar a mediana, não a primeira medição.

3

Provocar um erro de tipo em cada uma

Passar um texto onde o algoritmo espera número e registrar exatamente o que cada linguagem faz: mensagem de erro, resultado silencioso e absurdo, ou parada imediata.

4

Testar a soma de 0.1 com 0.2 nas duas

Registrar a saída literal e propor uma correção usando Decimal em Python.

5

Escrever a análise no repositório

Um arquivo COMPARATIVO.md com a tabela de tempos, o comportamento diante do erro de tipo e três parágrafos interpretando o resultado à luz dos três eixos

Entregável: COMPARATIVO.md e os dois arquivos de código, com commit feito ainda em aula

O QUE FICA DESTA AULA

Toda linguagem escolhe onde pagar a conta. Entender onde é o que permite escolher.

- Tipagem determina quando o erro aparece e quanto a linguagem improvisa quando os tipos não combinam.
- Execução determina o que acontece entre salvar o arquivo e ver o resultado, e explica o tempo de build e a velocidade.
- Memória determina quem devolve o espaço ao sistema, e é a origem histórica da maioria das falhas graves de segurança.
- Valor monetário não se guarda em ponto flutuante, em nenhuma linguagem.

Próxima aula: Paradigmas de programação, imperativo, funcional e orientado a objetos

Leitura para a próxima aula: tutorial oficial do Python, capítulo 9, seções 9.1 a 9.3. Trazer o domínio do projeto com as entidades já listadas.



Referências desta aula

LIVROS

SEBESTA, Robert W. Conceitos de Linguagens de Programação. 11. ed. Porto Alegre: Bookman, 2018.

Capítulo 5 sobre tipos de dados e capítulo 1 sobre métodos de implementação. É a referência direta desta aula.

RAMALHO, Luciano. Python Fluente. 2. ed. São Paulo: Novatec, 2023.

Autor brasileiro. Os capítulos sobre modelo de dados e referências explicam com precisão como Python trata objetos e memória.

NYSTROM, Robert. Crafting Interpreters. Genever Benning, 2021.

A parte II constrói um interpretador do zero e a parte III uma máquina virtual, tornando concreto tudo que foi visto no eixo de execução.

ARTIGOS, BLOGS E DOCUMENTAÇÃO

Joel Spolsky, The Law of Leaky Abstractions

<https://www.joelonsoftware.com/2002/11/11/the-law-of-leaky-abstractions/>

The Floating-Point Guide, o que todo programador deveria saber sobre ponto flutuante

<https://floating-point-gui.de/>

Python, aritmética de ponto flutuante, problemas e limitações

<https://docs.python.org/pt-br/3/tutorial/floatingpoint.html>

Chromium, Memory safety, origem dos defeitos graves de segurança

<https://www.chromium.org/Home/chromium-security/memory-safety/>

MDN Web Docs, conversão e coerção de tipos em JavaScript

https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Guide/Data_structures

Martin Fowler, Collection Pipeline, leitura para a aula 3

<https://martinfowler.com/articles/collection-pipeline/>